

Heron Simulator Architecture

The simulator owns Gazebo worlds, Heron spawn configuration, synthetic sensor providers, scenario entities, timing adapters, the drive plant, and hydrodynamic overrides. GRANDE composes the scenario; MARINER owns estimator, map, planner, and accepted drive; ORACLE owns mission semantics.

Gazebo ground truth supports sensor generation and labelled evaluation. It is not substituted for MARINER estimator feedback in navigation or control. Synthetic topics identify source and calibration eligibility.

Scenario YAML carries simulator world, spawn, entity, and map-bound facts consumed by ORACLE through GRANDE. That is integration convenience, not simulator ownership of mission policy. Entity fixtures are accepted only when they declare the supported schema, the active planning frame, and metres and radians as their units.

Scenario entity YAML and Gazebo geometry duplicate parts of the same scene, and no generator proves parity. Evaluation manifests identify both inputs so a mismatch remains an authoring defect rather than hidden uncertainty.

Heron Simulator Operation

The simulator normally starts through GRANDE:

```
cd ~/catkin_ws/heron_ws/src/grande/grande
python3 run.py bringup --mode sim --scenario harbor
```

The runner uses an isolated ROS master, normally port `11312`. Status, goal, and cancellation commands select that graph explicitly.

`heron_world.launch` selects world, rendering, time, and profile environment. `spawn_heron.launch` creates the vehicle and attaches sensor, timing, telemetry, and propulsion providers. GUI, headless, RViz, and software-rendering options change display behavior but not state or command meaning.

Simulated messages and propulsion command timeout use ROS simulation time. MARINER uses ROS time for selected simulation control loops and monotonic time on hardware; some cancellation barriers use wall timers. Runner process supervision and shutdown use wall time and remain scoped to child processes and the isolated master it started.

`open_water.world` references external `ned_frame` and `sand_heightmap` models. Runtime output retains scenario, configuration, commit, and synthetic-source provenance.

Heron Simulator Sensors

The simulator publishes device-like observations for GRANDE sensor classes. The goal is interface and timing fidelity within a declared synthetic model, not a claim that Gazebo reproduces full physical error distributions.

`urdf/sensors.urdf.xacro` defines simulated sensor additions and imports the canonical sensor poses from IG Handle. Frame names, geometry, and topic meanings therefore remain aligned with the physical sensor contract. Gazebo measurement, noise, rendering, and transport behavior are simulator-owned artifacts.

Gazebo plugins provide LiDAR, camera, IMU, and vehicle observations. `sim_ig_timing.py` publishes IG Handle-style timing surfaces. `sim_sense.py` publishes explicitly synthetic Heron telemetry tied to the selected propulsion plant.

Multibeam and Ping360 providers translate Gazebo rays into their canonical profile meanings. They represent geometry and transport, not acoustic propagation, transducer response, multipath, turbidity, or real latency. The DT100 transport publishes only `dt100_link` with extrinsic revision `dt100-seed-2026-08-11-v1`; Ping360 publishes only `ping360_link` with revision `ping360-seed-2026-08-11-v1`. Both revisions are provisional, unmeasured seeds. Each provider rejects a Gazebo cloud whose source frame differs from its configured frame. The strict 83P packet and normalized Ping360 profile carry the matching revision, and the Ping360 profile identity hash also binds the provider, model, frame, and revision. Provider and model values

match the hardware-facing contracts. `gazebo://dt100` plus the DT100 launch contract and the Ping360 profile's explicit `synthetic=true` field retain synthetic-source provenance; downstream consumers do not infer it from provider names or free-form validity text.

Ground truth drives synthetic providers, but navigation consumes MARINER's estimator surface. Synthetic provenance remains attached even when downstream interfaces match physical acquisition.

Descriptor-driven range marker

RANGE_AID owns the canonical Orion marker descriptor and its marker-instance record. The simulator does not copy either geometry or placement. At launch, `acoustic_marker_model.py` consumes RANGE_AID's canonical validation of finite dimensions, non-overlap, three-dimensional span, and distinct sphere signatures. It then applies only SDF identifier and explicit provisional opt-in gates before emitting stable SDF. `spawn_acoustic_marker.py` checks the instance against the same ID and revision before calling Gazebo's model-spawn service.

The provisional marker contains five unequal, asymmetrically placed spheres and four thin support struts. Sphere and strut `laser_retro` fields are deterministic Gazebo ray-intensity surrogates only. They do not model target strength, frequency response, beam pattern, multipath, absorption, scattering, or underwater material response.

`range_marker_pool.world` includes the tank and visual water surface but omits the legacy `tank_targets` model. Its scenario points to exactly one RANGE_AID descriptor and provisional instance at map/world position `(0, 0, -1.50) m` with 160 degrees yaw. Ground truth is available for synthetic scoring only; it is not fed to marker detection or estimation.

The DT100 proxy provides one vertical cross-track slice per ping, and the Ping360 proxy provides sequential horizontal profiles. The deep marker is inside the DT100 cross-track field but requires vehicle motion for multiple along-track views. A single slice cannot establish an unrestricted six-degree-of-freedom marker pose. The current Ping360 ray model has one zero-thickness horizontal plane and cannot observe this same deep instance; it is a planar negative control in this scenario, not an alternative full-pose result. Complete pose evaluation must use time-aligned DT100 motion and multiple views, with explicit degeneracy and observability reporting downstream.

Heron Simulator Scenarios

Scenarios bind a Gazebo environment, initial vehicle state, semantic entities, and integration settings into a reproducible configuration.

The tank world centers its tank, water-surface, and target models on the spawn/map origin so symmetric positive and negative navigation coordinates stay inside the controlled water volume. The harbor profile supports mapping, navigation, exploration, and inspection. Open water references external `ned_frame` and `sand_heightmap` models.

The exploration arena is a synthetic, logically bounded, reduced-load sensing and navigation integration scenario. It uses three finite static structures: `exploration_wall`, `exploration_return_wall`, and `exploration_south_breakwater`. They provide observable planar geometry and a bounded decision-directed routing area while ORACLE builds a fresh live map. Entity YAML remains simulator/evaluation reference truth in mapless runs rather than a planner-visible semantic prior. Success in this arena exercises software interfaces and the configured simulator plant; it does not validate physical sensor error, hydrodynamics, or vehicle performance.

Scenario YAML supplies initial pose, entities, world selection, offsets, and environmental map bounds. Mission and navigation policy comes from named GRANDE runtime profiles rather than simulator configuration.

Semantic records describe what ORACLE may discover; Gazebo files describe collision/visual geometry. No generator currently guarantees parity. Recorded runs identify both scenario and world inputs.

Scenario map bounds flow to ORACLE as environmental input. A result is meaningful only with its world, vehicle profile, sensor configuration, initialization, controller, mission profile, and repository state.

Heron Simulator Propulsion

The propulsion path converts MARINER's normalized left/right drive request into Gazebo forces and synthetic actuator telemetry. It is a simulator plant, not a physical-Heron calibration.

`drive_to_thrusters.py` represents direction, deadband, saturation, voltage scaling, lag, slew, and reversal blanking and publishes synthetic PWM, RPM, current, voltage, thrust, and status. `sim_sense.py` uses the same plant state so electrical surfaces remain internally consistent.

The standard plant's full-command scale uses the legacy Heron simulator's [trial-run thrust table](#): 33.6 N forward and 19.88 N reverse per thruster. The local July 20 electrical profile informs side- and direction-specific onset and current proxies, not force. Payload, inflow, RPM, thrust, and hydrodynamic coefficients were not jointly identified, so these remain provisional simulation parameters rather than a calibration of the IG Handle Heron.

Vehicle mass, inertia, fluid density, damping, and thruster placement determine motion response. Overlays are part of recorded simulator configuration and do not change the real Heron path.

Command-to-current, current-to-thrust, thrust-to-motion, and closed-loop control remain separate relationships. Simulation can expose asymmetry, reversal, saturation, and timing but cannot establish the physical force law.

The defensible description is **physics-based simulation with provisional, partly data-informed actuator parameterization**. Historical data informed some electrical/asymmetry choices, but mass, inertia, damping, absolute force, and water-relative disturbance are not jointly identified or physically validated. Accordingly, this simulator is the most integrated maintained software plant, not an identified data-based model of the real Heron.